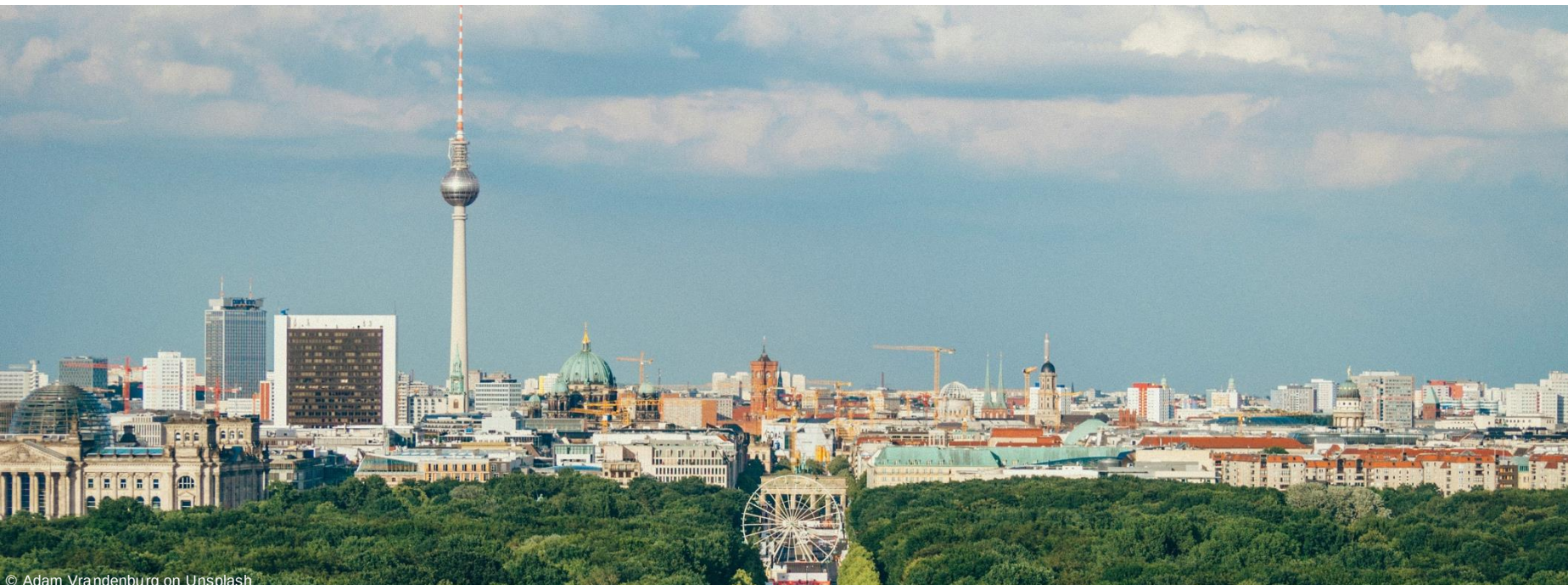


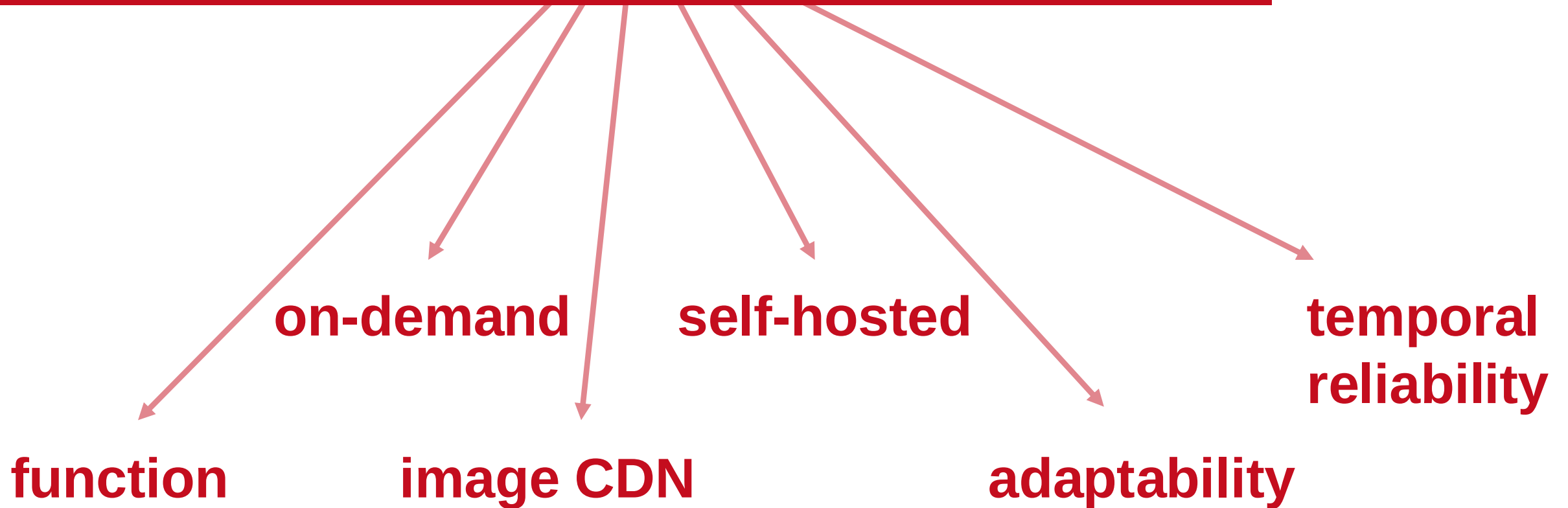


Practical FaaS Performance Variability Implications in the Serverless Image Optimization Scenario

Piotr Witkowski | Scalable Software Systems | February 14, 2025



Practical FaaS Performance Variability Implications in the Serverless Image Optimization Scenario

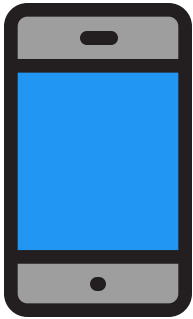


BACKGROUND

ARCHITECTURE

RESULTS

What is Image Optimization?



“1 image please!”



“1 image please!”



What is Image Optimization?

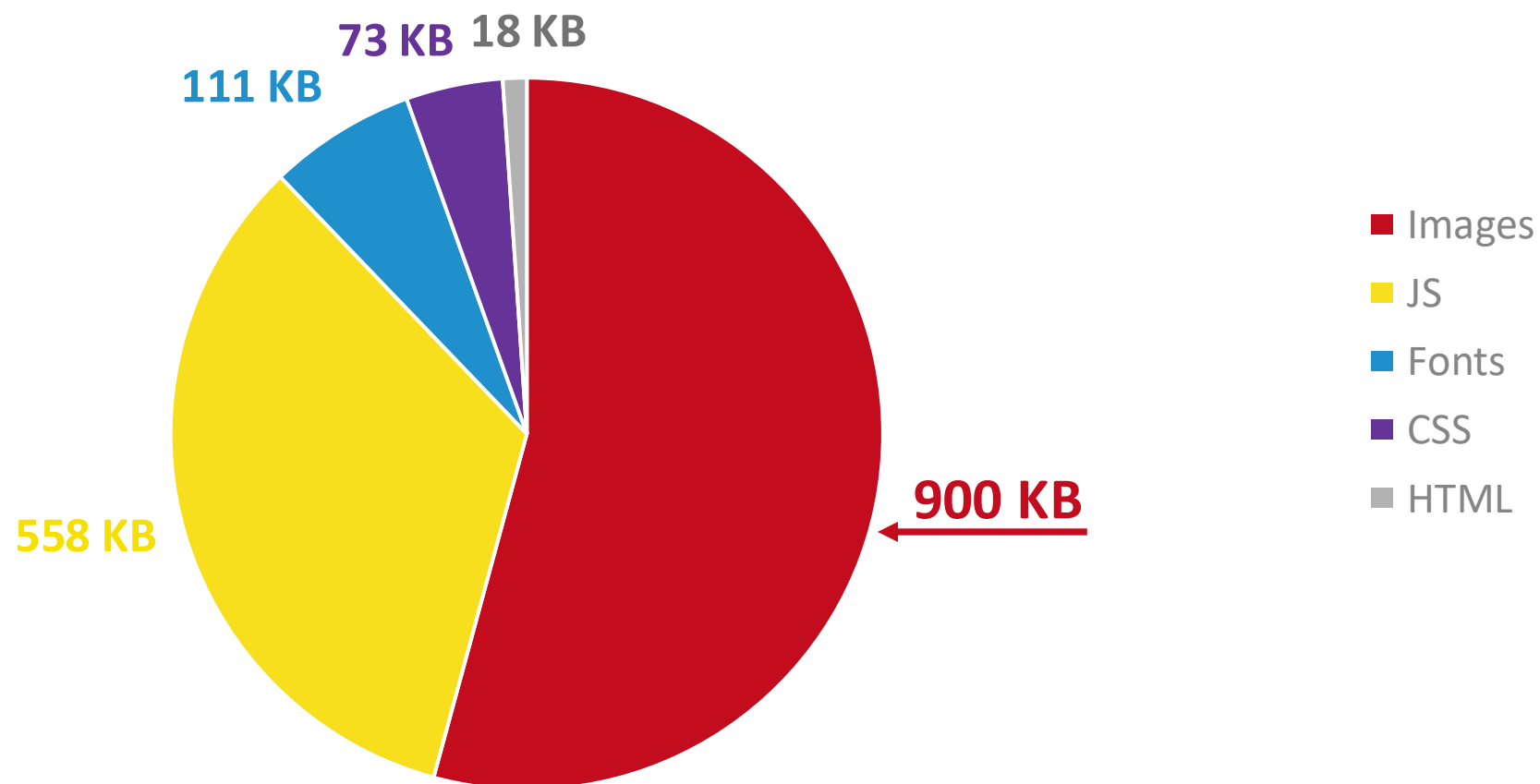


Screen	Small, limited by portability	Large, typically stationary
Networking	Highly variable, typically wireless	Reliable, wireless or wired connection
Compute	Limited battery power & cooling	Efficient cooling for sustained load

Idea: provide different clients with optimized assets



Median homepage weight, by content type



Source: <https://almanac.httparchive.org/en/2024/page-weight>

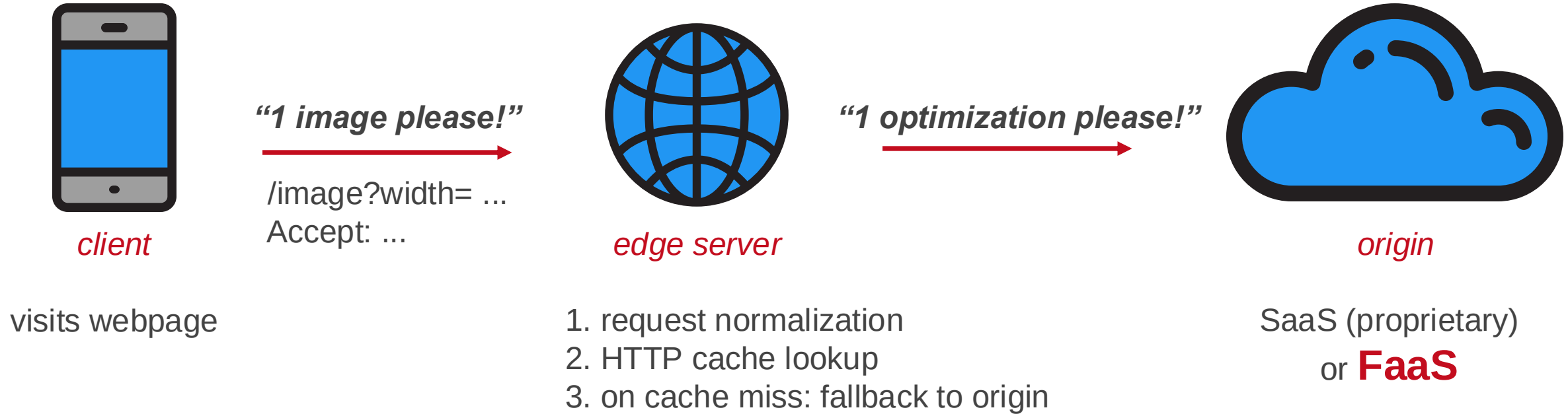
HTTP Image Requests

Simple request GET /image.jpg

Older browser GET /image.jpg?format=auto&width=**1200**
Desktop Accept: image/gif, image/**jpeg**, image/pjpeg, */*

Modern browser GET /image.jpg?format=auto&width=**320**
Mobile Accept: image/**avif**, image/**webp**, image/apng, image/*, */*;q=0.8

Image CDNs



BACKGROUND

ARCHITECTURE

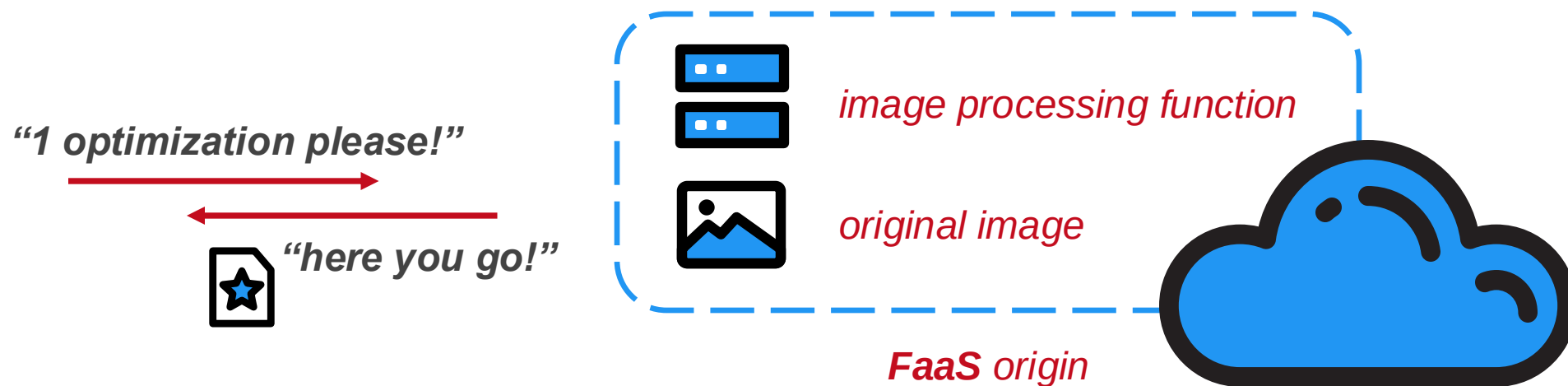
RESULTS

Serverless Image Optimization with FaaS



Image optimization steps:

1. **processing setup**, incl. cold start and loading original image into memory
2. **image processing** – *I am investigating performance variability of this step*
3. **HTTP response**, and optionally long-term caching of optimized assets



Parametrizing Optimization with Function Variants

Parameter	Values	
Provider	AWS Lambda / Google Cloud Run functions	2
Input image	“landscape” / “portrait”	2
Output format	JPEG / AVIF / WebP	3
Output width	640 / 1080 / 1920 pixels	3
Assigned vCPUs (memory)	0.5 / 1 / 2 (885 / 1769 / 3538 MB)	3
		$2^2 * 3^3 = 108$ variants

Side notes:

- AWS Lambda allocates CPU time proportionally to assigned memory
- Google Cloud Run offers independent settings for assigned vCPUs and memory
- **Every variant deployed as separate function** to minimize interference between resources

Test Images



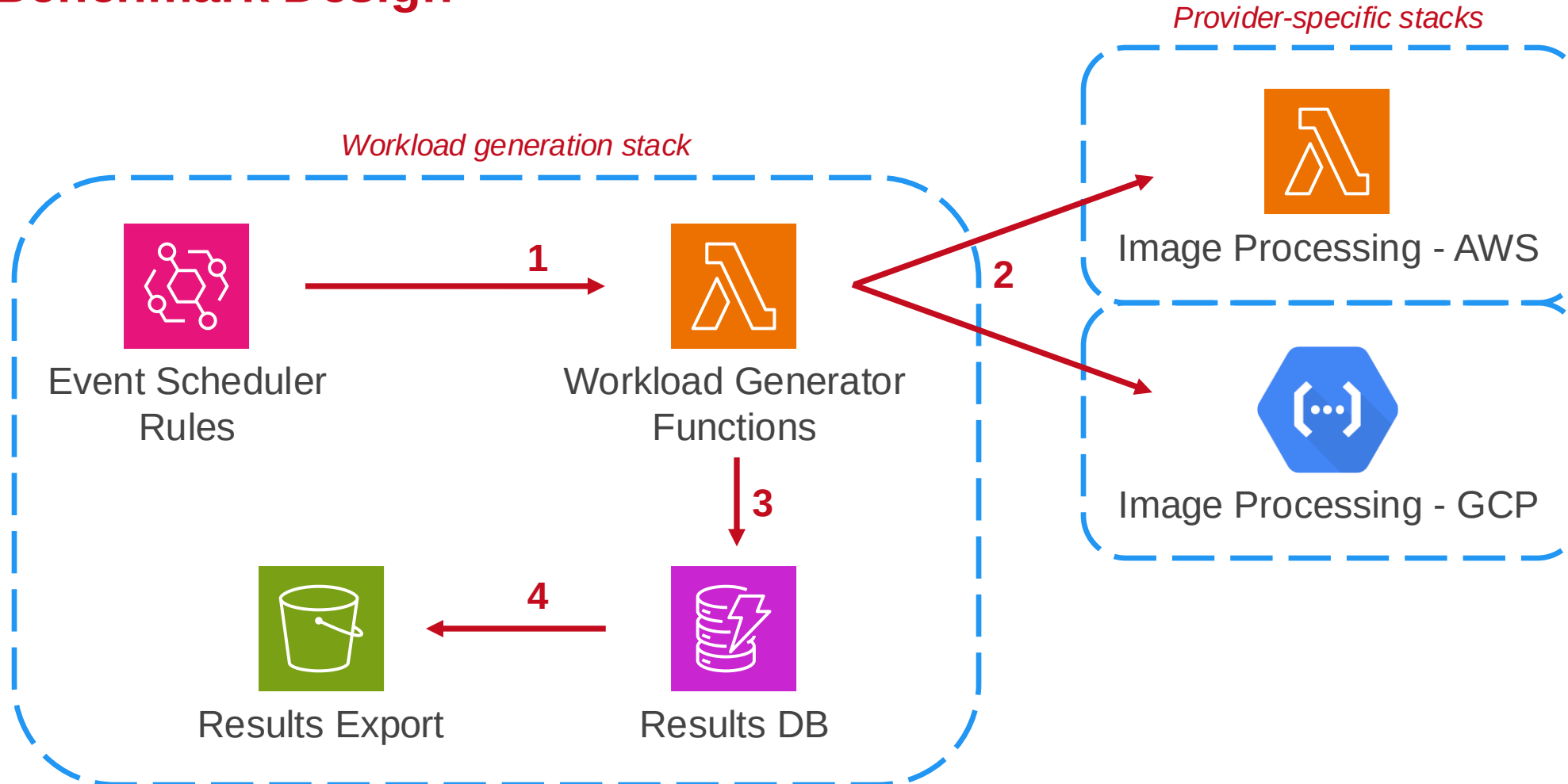
landscape
JPEG, 23.2MB
8256 × 5504 pixels, 45MP



portrait
JPEG, 9.9MB
3456 × 5184 pixels, 18MP



Benchmark Design



Dataset & Performance Variability Categories

108 functions executed every 5 minutes, for 2 months = **2M executions**

108 functions * 12 times /hour * 24 hours /day * 62 days (Dec, Jan, Feb)

= 1,928,448 (*best case*)

- Cold starts & environment reuse? depends on the provider...
- Missed executions? yes, a few...
- Temporal variability categories:

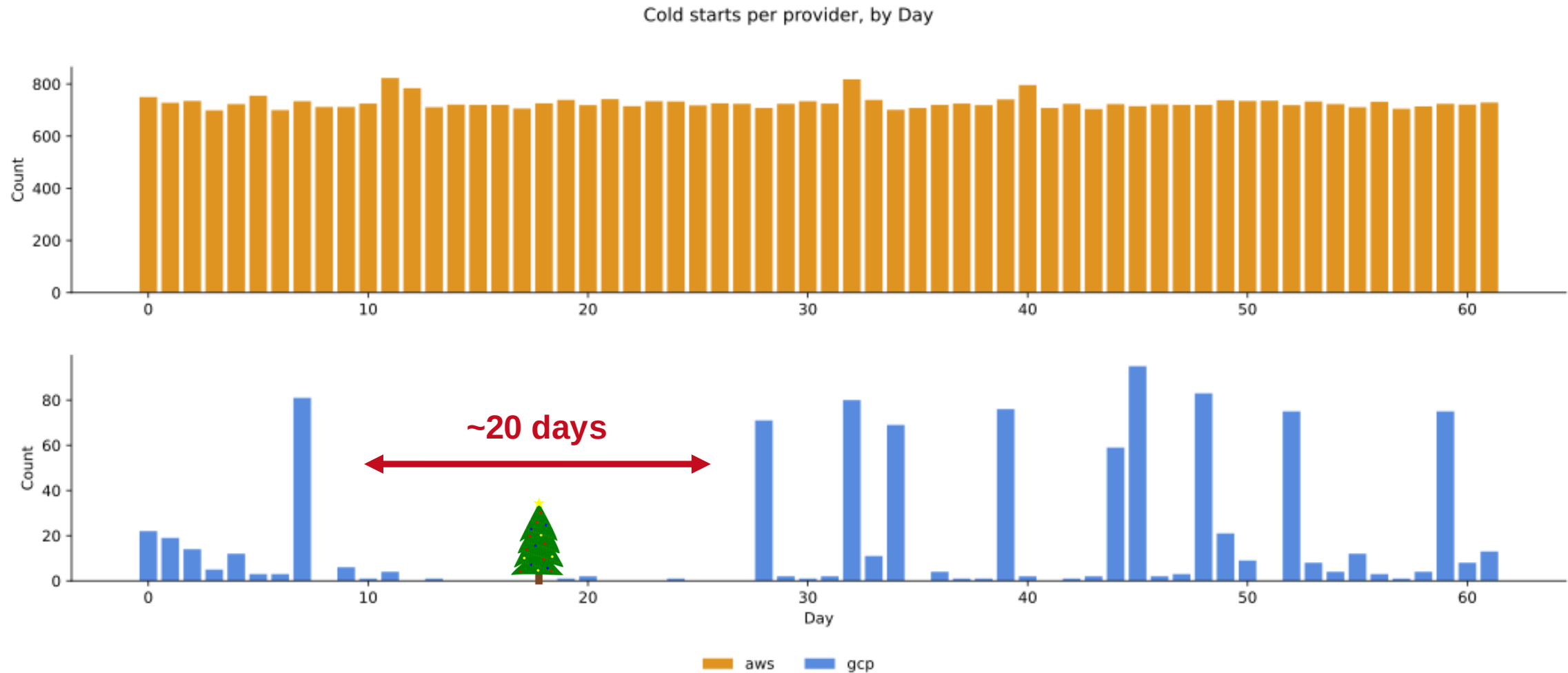
	Day	Week
Day	Day of Day (single value)	Day of Week 7 days, Mon – Sun
Hour	Hour of Day 24 hours	Hour of Week 168 hours
Minute	Minute of Day 1440 minutes	Minute of Week (too granular)

BACKGROUND

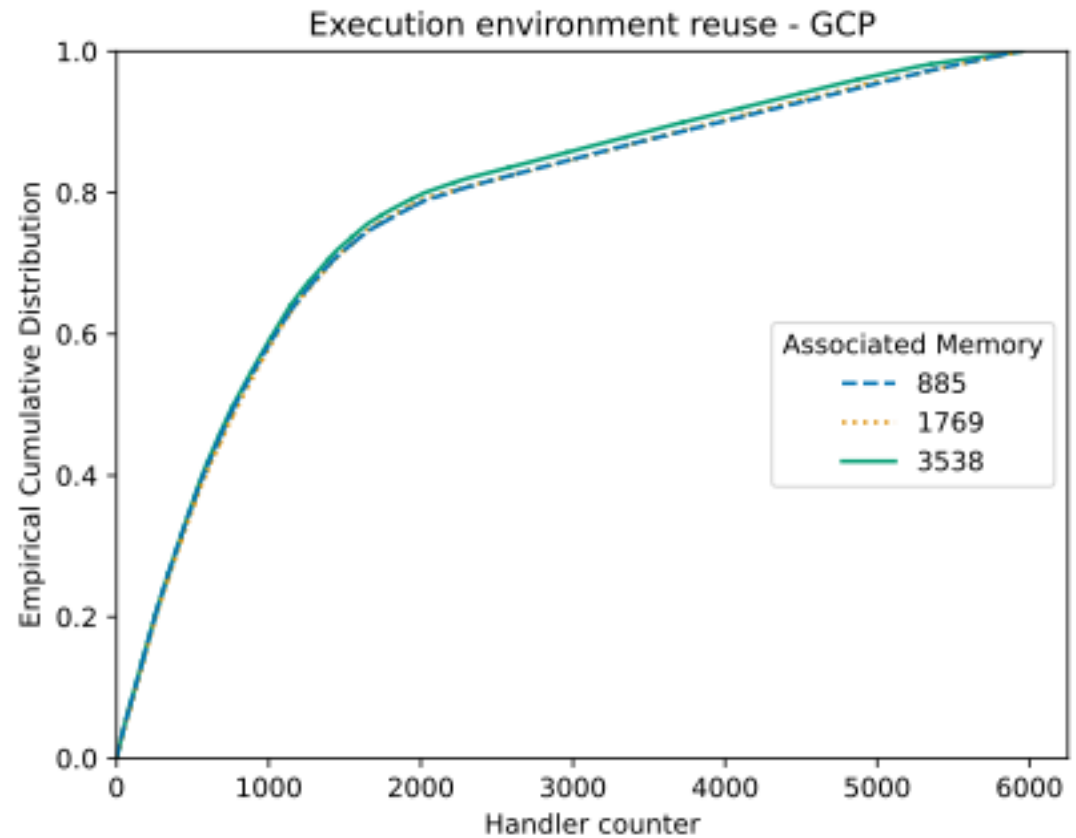
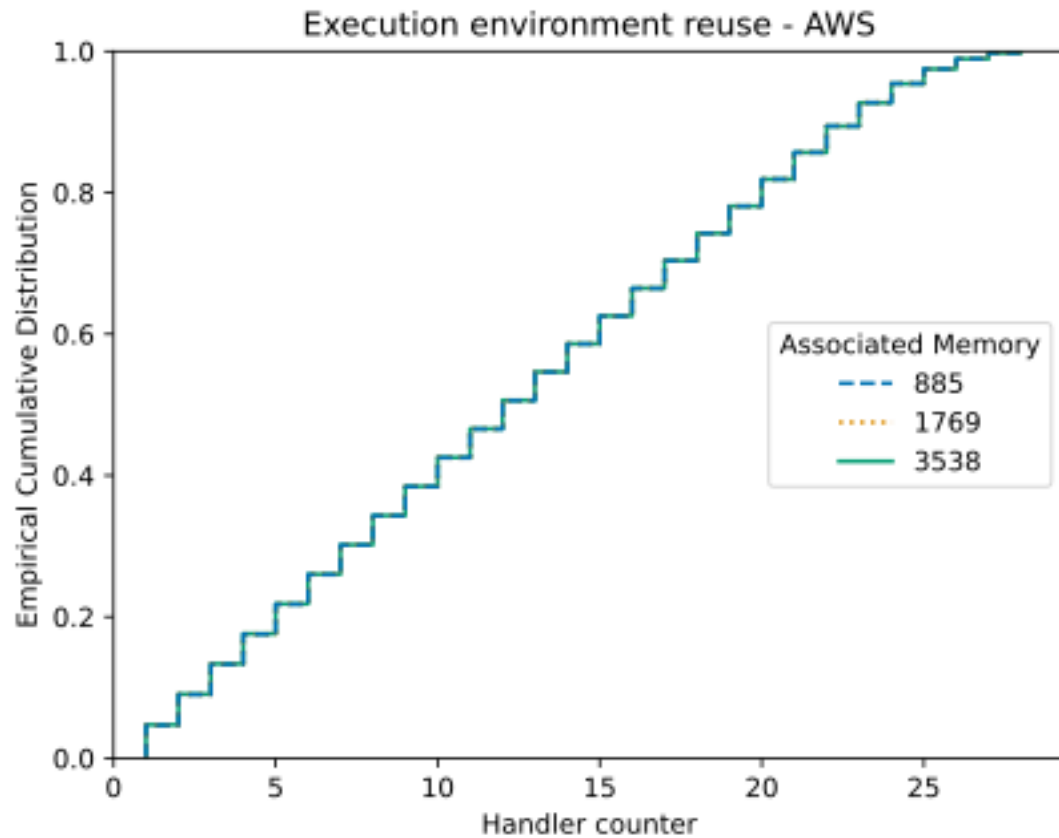
ARCHITECTURE

RESULTS

Cold Starts



Environment Reuse

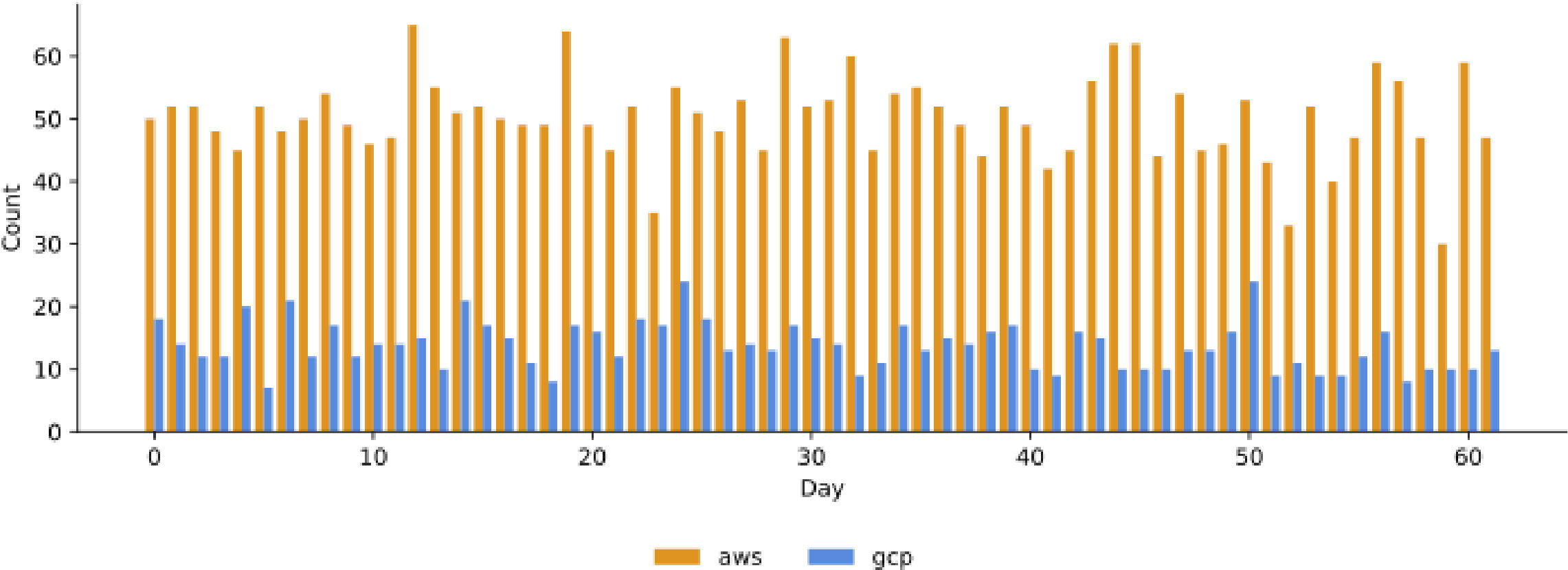


Missed Executions

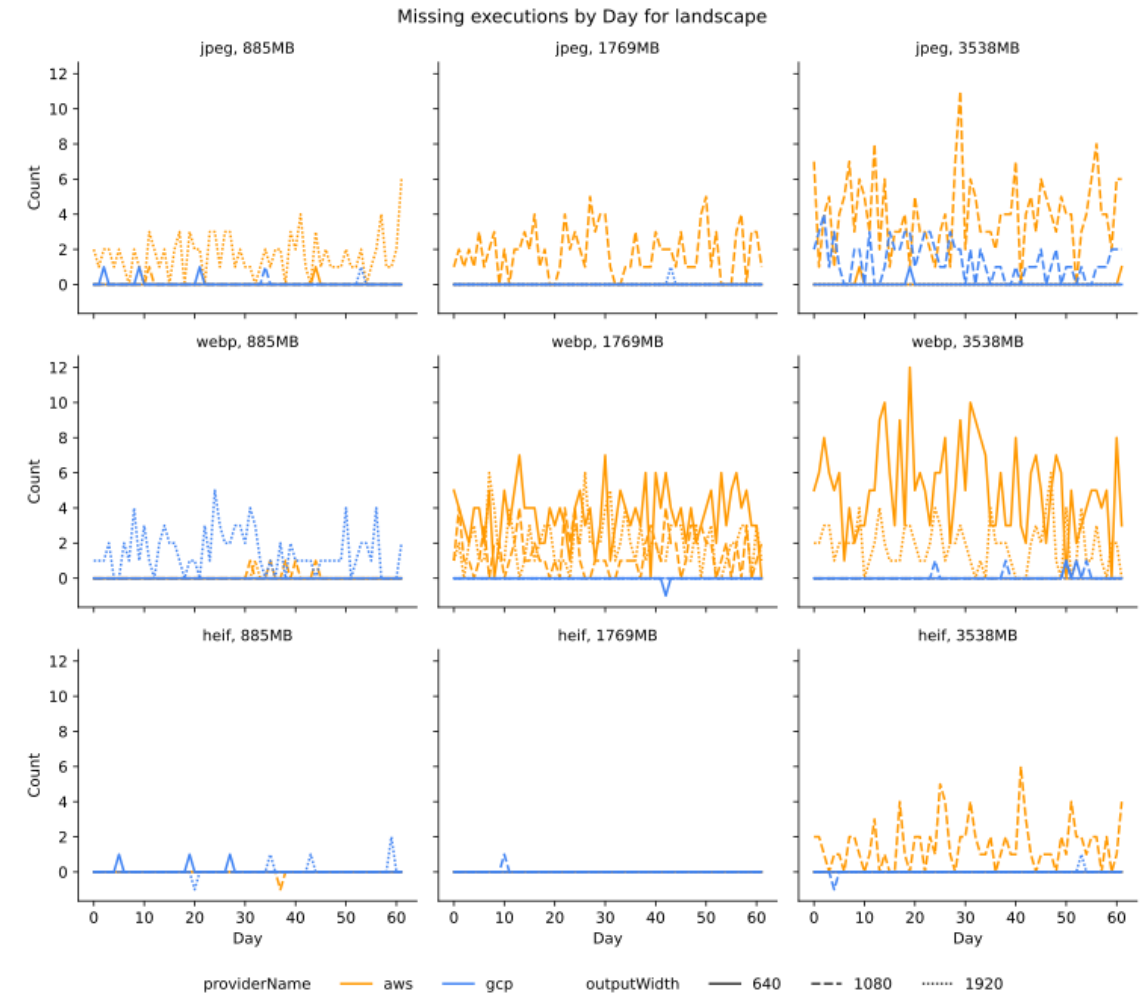
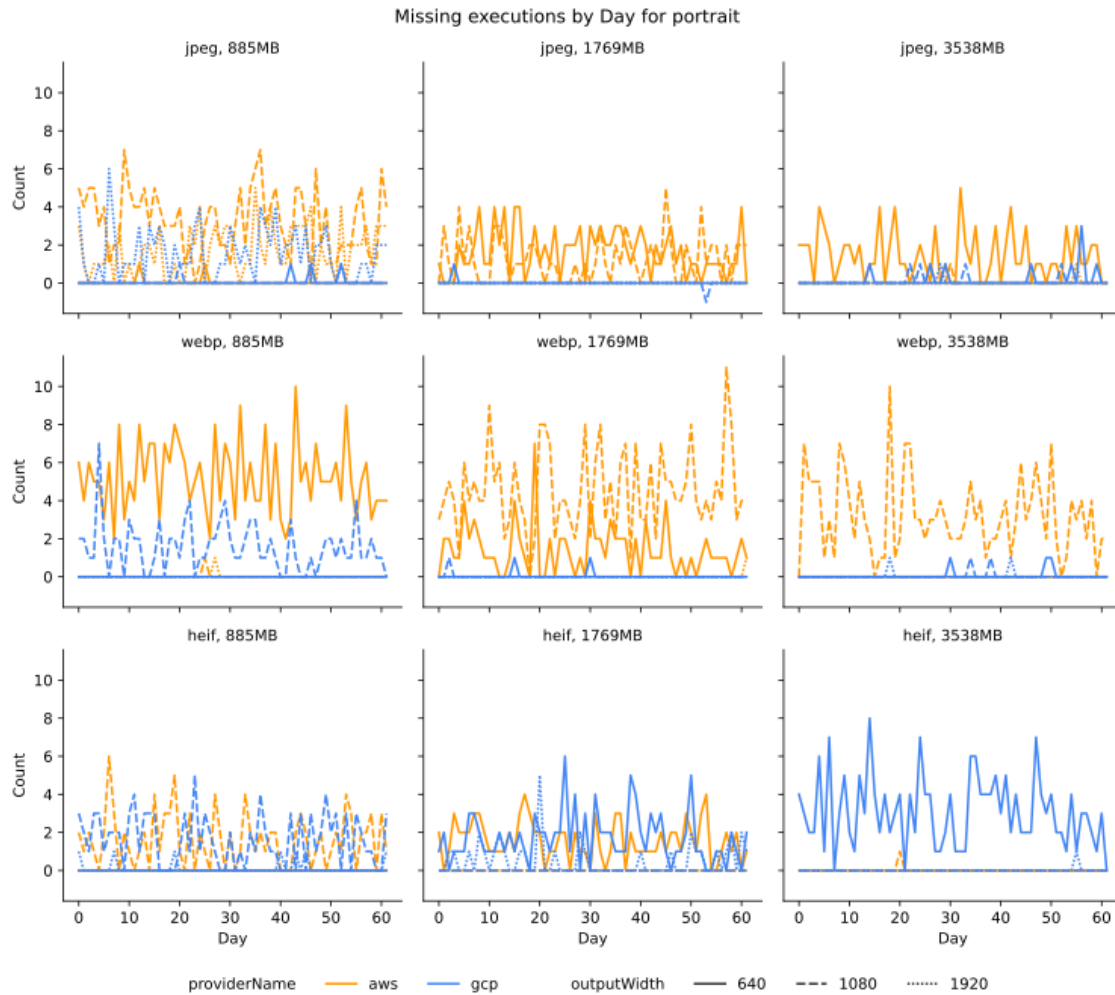


Expected: 1928448
Actual: 1924484, **99.79%**
Missed: 3964, **0.21%**

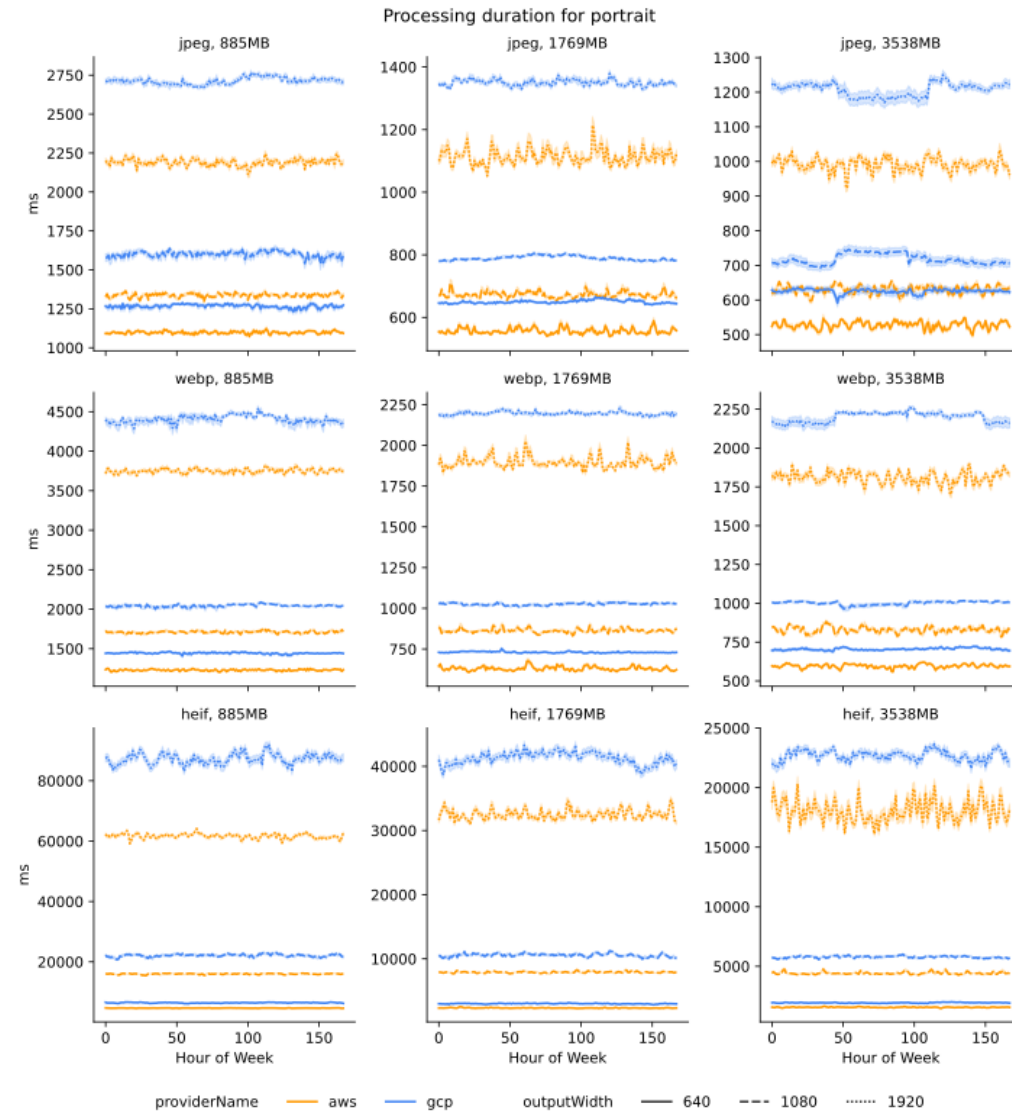
Expected, per day: 31104
Actual, per day: 31040.06
Missed, per day: 63.94



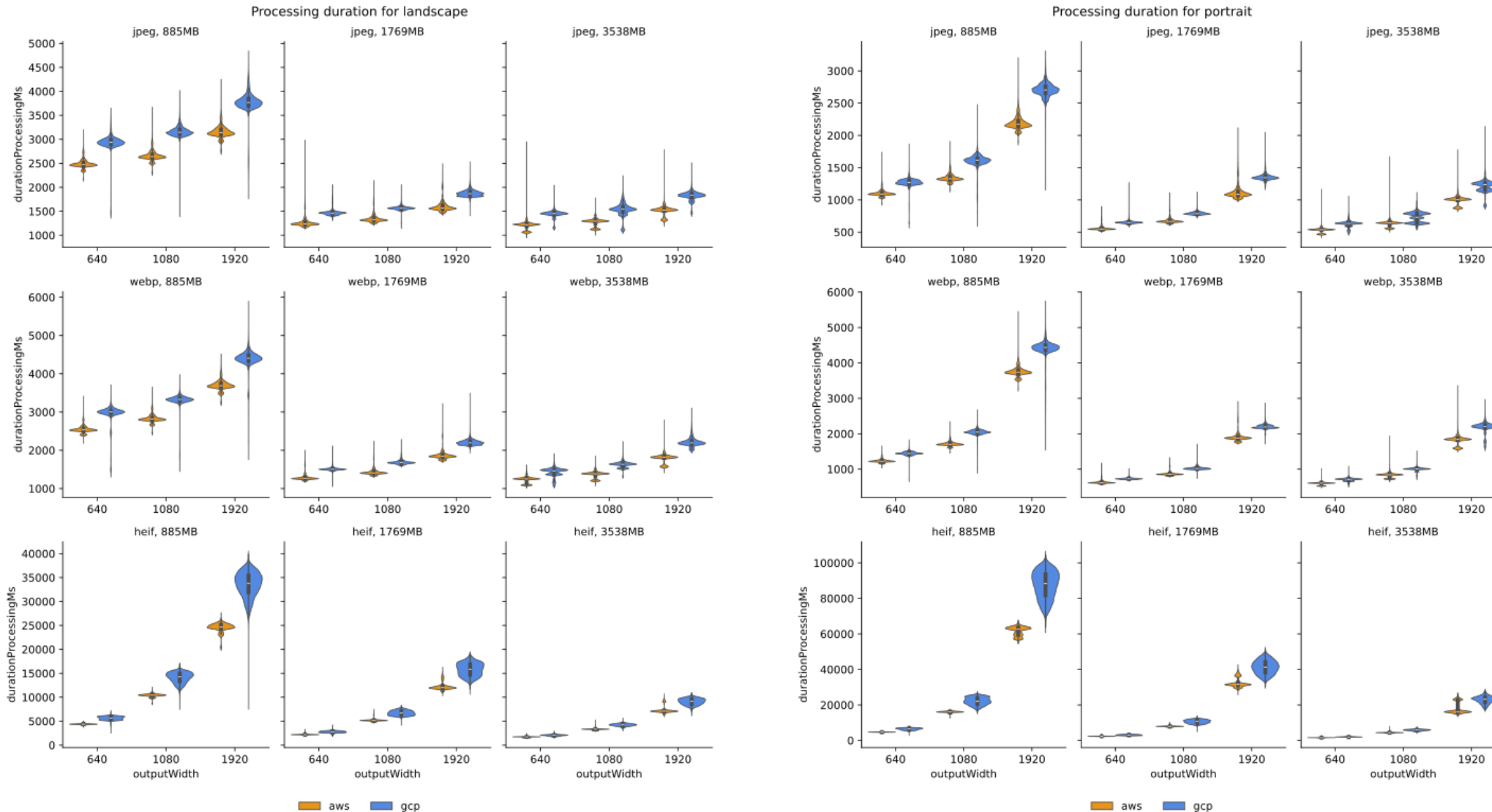
Missed Executions, by function (all 108 functions)



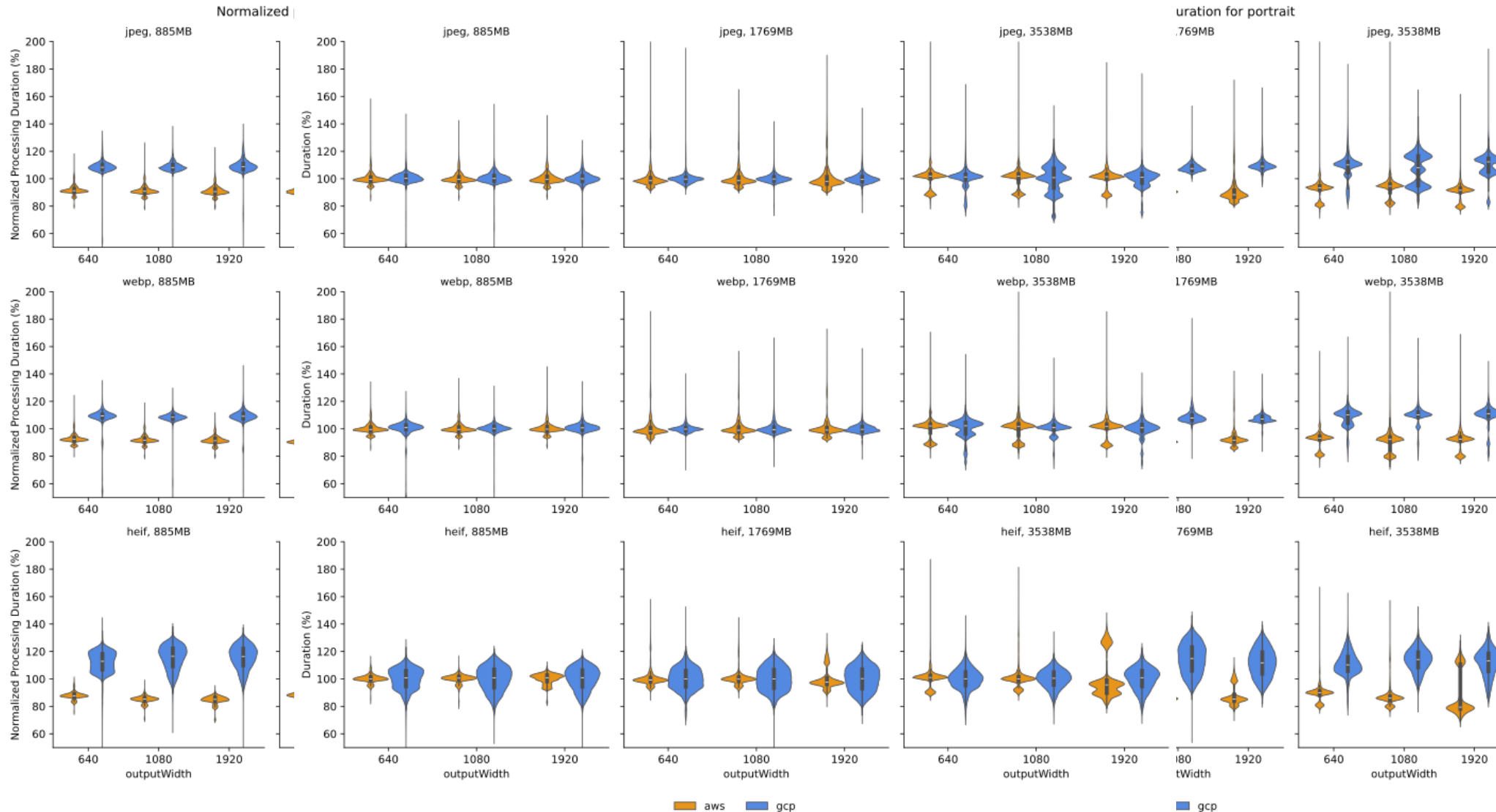
Naive first attempts



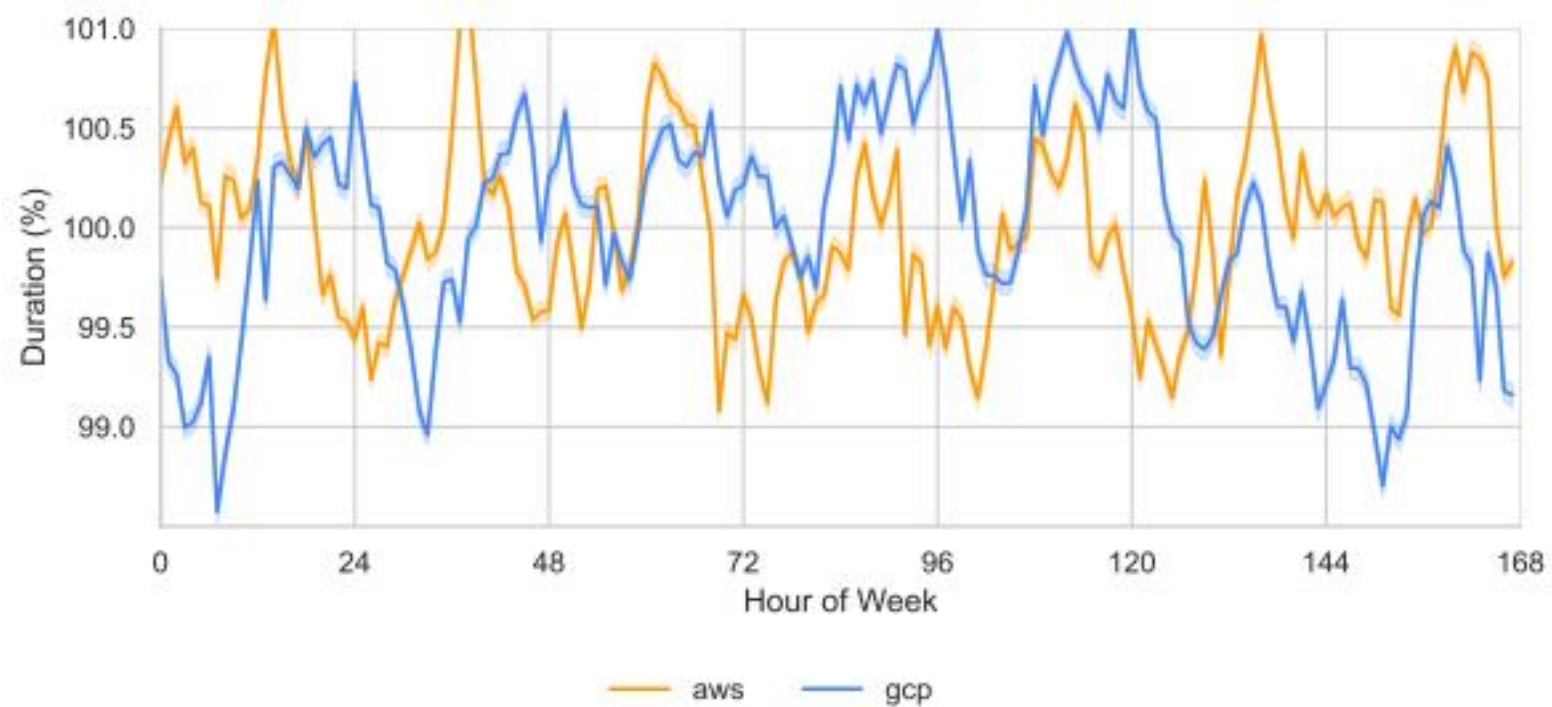
Processing duration distributions: absolute



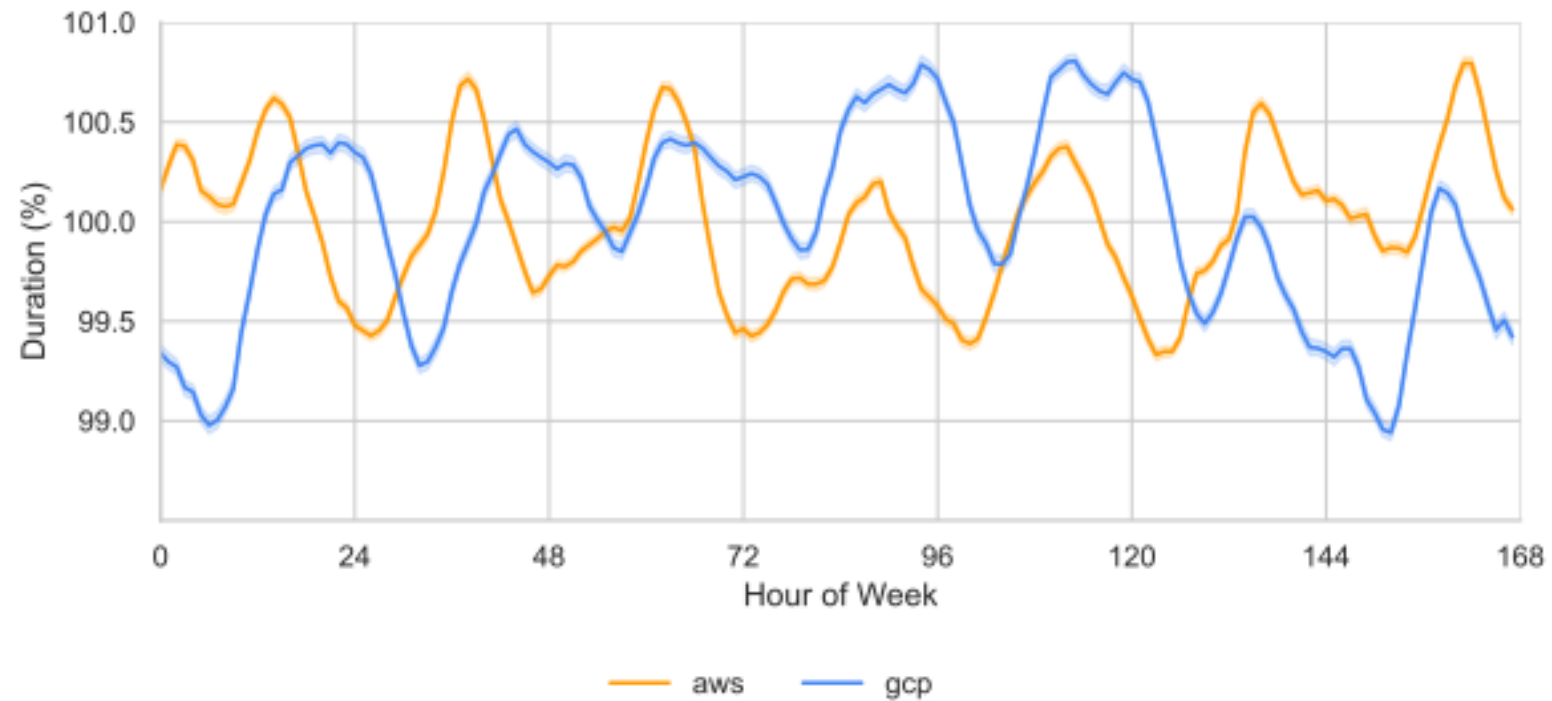
Processing duration distributions: relative



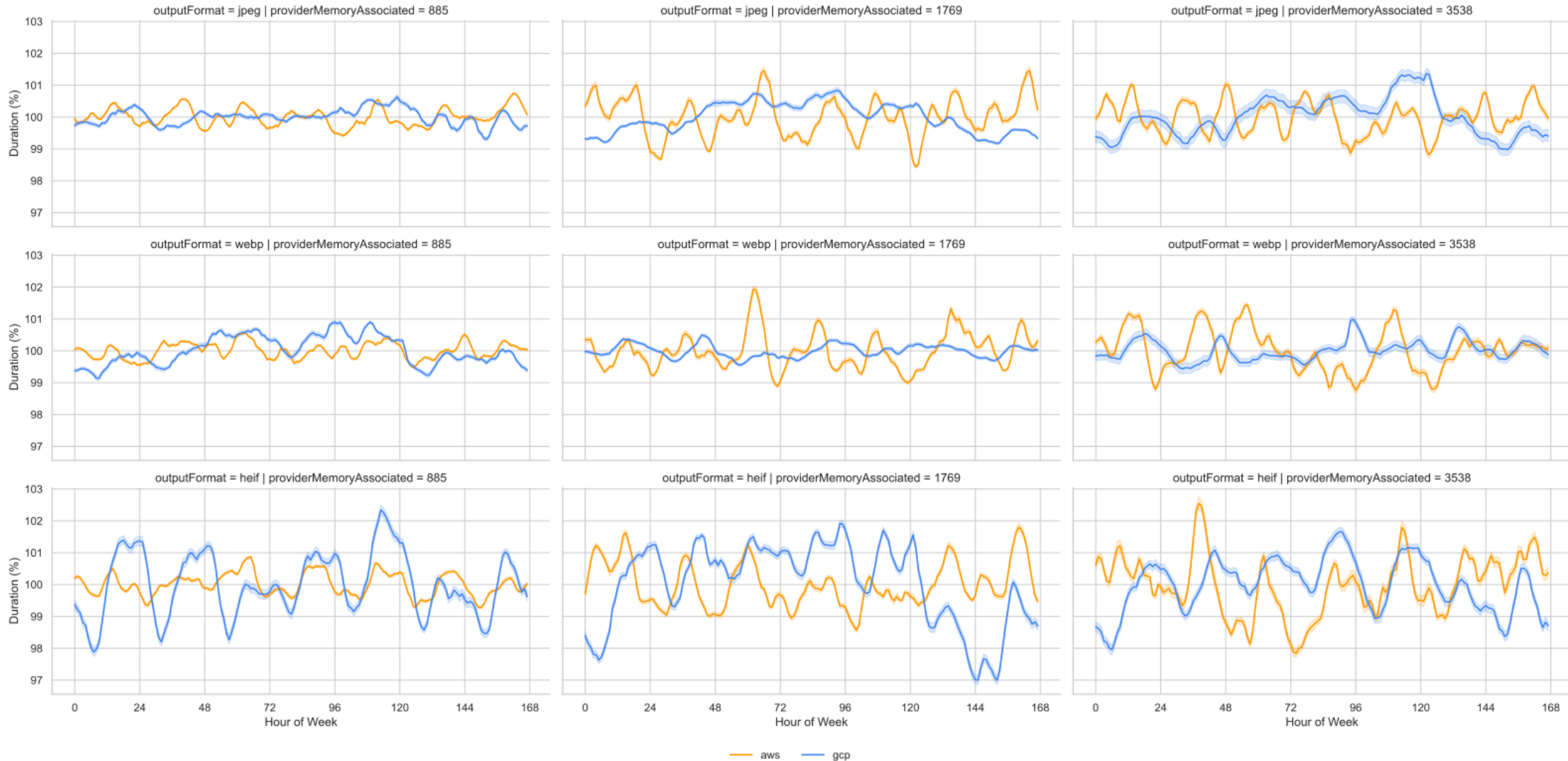
By Hour of Week – normalized & aggregated



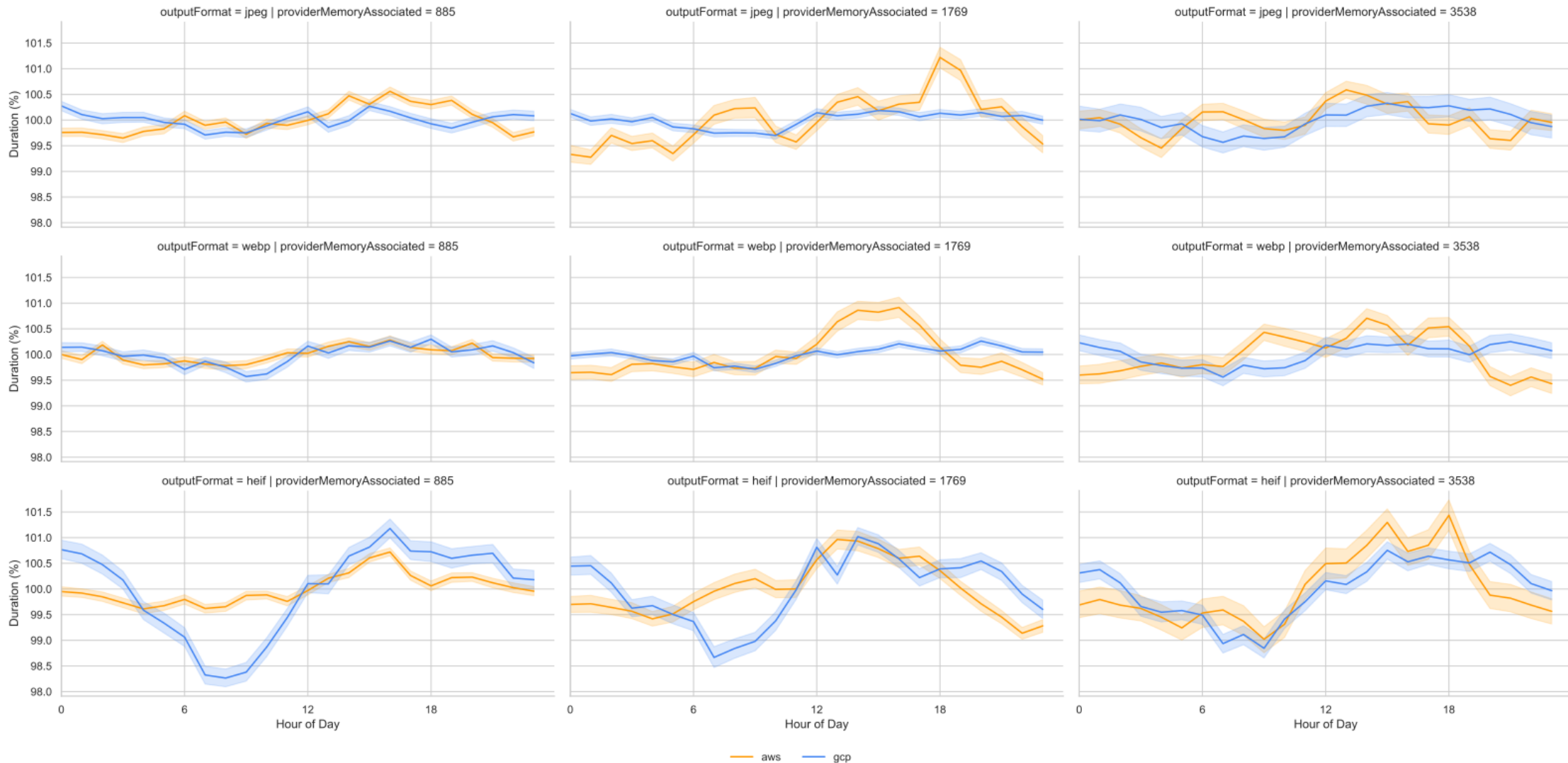
By Hour of Week – with rolling average



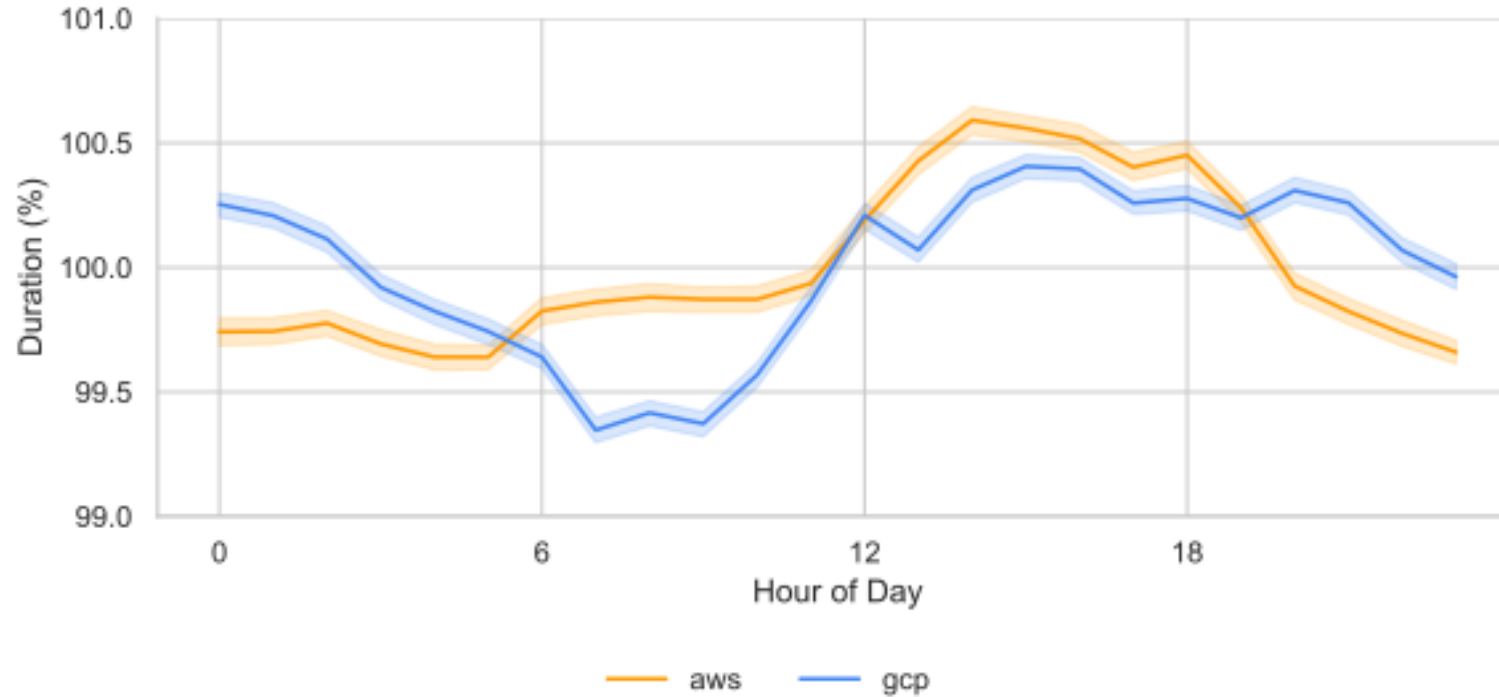
By Hour of Week – granular view



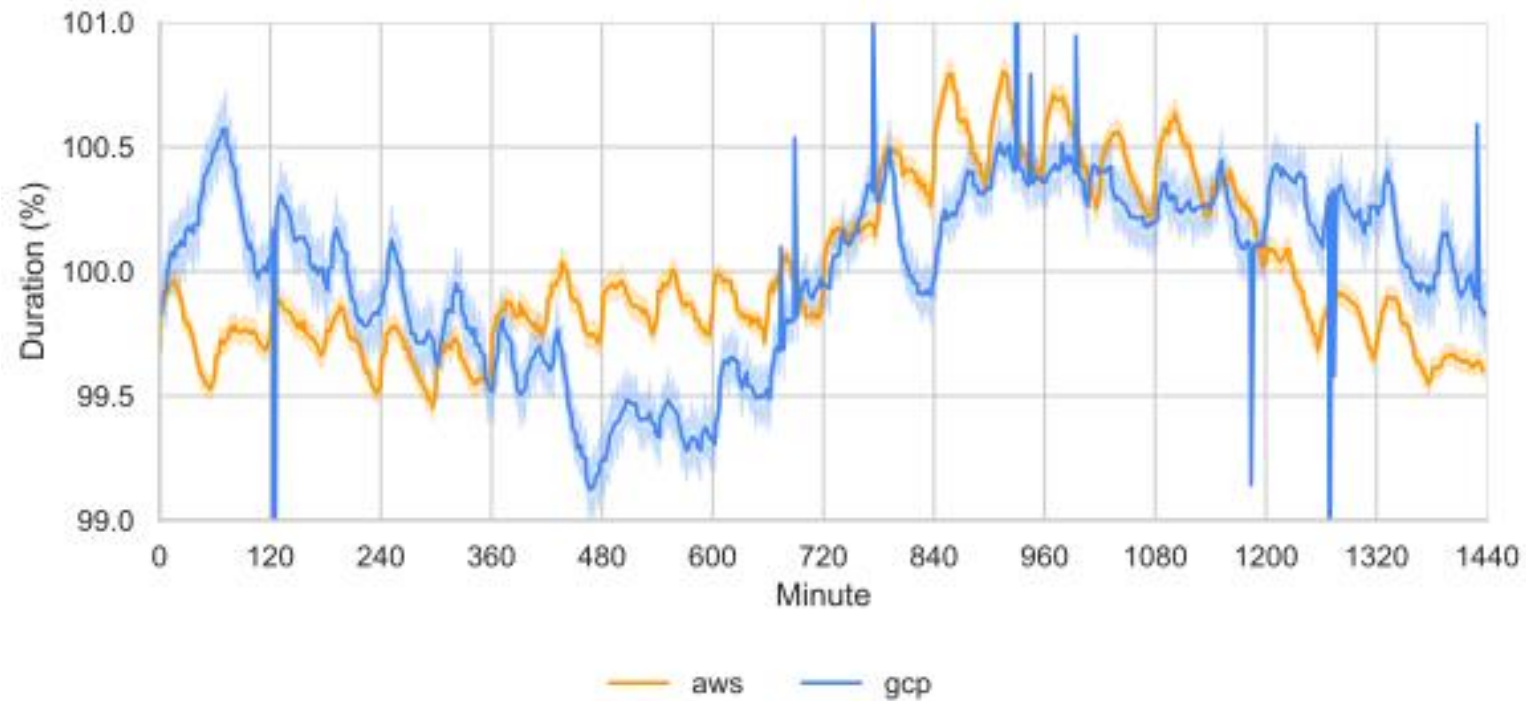
Hour of Day



Hour of Day



Minute of Day



Summary & Takeaways – so far



Confirmed variability:

- | | |
|-------------------------------------|--|
| • hour of week (<i>how</i>) | up to ~ 2% overall |
| • hour of day (<i>hod</i>) | up to ~ 1% overall , up to 3% for specific functions |
| • minute of hour (<i>mod</i>) | up to ~ 0.5% |
| • weekday vs weekend (<i>how</i>) | only on GCP? |
| • cold vs hot start | mostly on AWS?, not discussed today |

More observations:

- cold start behavior & missed executions differ by provider
- detailed error tracking could provide further details about missed executions
- adding 2nd CPU does not result in 2x faster optimization on tested FaaS platforms
- *1 vCPU* is not universal across studied FaaS providers



Thank you!